

Oleg Fatkhiev <tender-bum@yandex-team.ru>, <brickmen75@gmail.com>
Antony Polukhin, Yandex.Taxi Ltd, <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2018-10-02

Variant direct comparisons

I. Motivation

In the current working draft [N4762] variant could be compared only with variant of the same type. There's no operator to compare variant with arbitrary types:

```
#include <array>
#include <variant>

int main() {
    using arr_t = std::array<int, 10>;
    using var_t = std::variant<int, arr_t>;

    var_t v;
    arr_t a = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
    return v == a; // Ill formed.
}
```

This paper attempts to solve that without adding ambiguous comparisons:

```
#include <array>
#include <variant>

int main() {
    using arr_t = std::array<int, 10>;
    using var_t = std::variant<int, arr_t>;

    var_t v;
    arr_t a = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
    v == a; // OK

    std::variant<int, double> v2(1.0);
    // Does the user wish to know that variant holds some alternative that is less than 0.0, or that variant holds an `double` that is 1
    // Some people expect `true` as the variant holds an integer, not a double. Other expect `false`, because 1.0 is greater than 0.
    v2 < 0.0; // Ill formed.

    std::variant<std::variant<int>> v3(1);
    // Does the user wish to know that variant holds an integer deep within, or is that some typo?
    v3 == 0; // Ill formed.
}
```

In other words, we propose to add comparisons that have single interpretation. To achieve that we allow comparisons for cases when only a single alternative is comparable with the non-variant type and mimic the behavior of two variant comparisons if variant is `valueless_by_exception()`.

II. Impact on the Standard

This proposal is a pure library extension and it does not break the existing code and does not degrade performance. It does not require any changes in the core language and could be implemented in the standard C++.

III. Wording

Add the following to the [variant.syn]

```
...

template<class... Types>
constexpr bool operator<=(const variant<Types...>&, const variant<Types...>&);
template<class... Types>
constexpr bool operator>=(const variant<Types...>&, const variant<Types...>&);

template<class... Types, class T>
constexpr auto operator<=>(const variant<Types...>&, const T&);
template<class T, class... Types>
constexpr auto operator<=>(const T&, const variant<Types...>&);
```

Add the following to the end of [variant.relops]

```
template<class... Types, class T>
constexpr auto operator<=>(const variant<Types...>& v, const T& t);
```

Constraints: `get<i>(v) <=> t` is well-formed expression for only one *i*; *T* and `Types...` are not a specialization of `variant`.

Returns: `i <=> variant_npos` if `v.valueless_by_exception()`; otherwise `v.index() <=> i` if the result of that expressions is not equal to `0`, otherwise `get<i>(v) <=> t` with *i* being the index of the type for which the expression `get<i>(v) <=> t` is well-formed.

```
template<class T, class... Types>
constexpr auto operator<=>(const T& t, const variant<Types...>& v);
```

Constraints: `t <=> get<i>(v)` is well-formed expression for only one *i*; *T* and `Types...` are not a specialization of `variant`.

Returns: `variant_npos <=> i` if `v.valueless_by_exception()`; otherwise `i <=> v.index()` if the result of that expressions is not equal to `0`, otherwise `t <=> get<i>(v)` with *i* being the index of the type for which the expression `t <=> get<i>(v)` is well-formed.

Add a row into the "Standard library feature-test macros" table [support.limits.general]:

<code>__cpp_lib_variant_direct_cmp</code>	<code>201811</code>	<code><variant></code>
---	---------------------	------------------------------